

File Provisioner

In **Terraform**, a **file provisioner** is a mechanism used to copy files from the machine running Terraform (usually your local system or CI/CD runner) to a remote resource, typically a virtual machine (VM) created by Terraform. It's part of the broader **provisioner** system, which allows you to execute scripts or perform configuration tasks on resources after they're created.

File Provisioner Syntax

```
resource "aws_instance" "example" {  
  ami      = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"
```

```
  provisioner "file" {  
    source      = "app.conf"  
    destination = "/etc/app.conf"
```

```
    connection {  
      type      = "ssh"  
      user      = "ec2-user"  
      private_key = file("~/ssh/id_rsa")  
      host      = self.public_ip  
    }  
  }  
}
```

Key Attributes

- **source**: Path to the file or directory on the local machine.
- **destination**: Path on the remote machine where the file should be copied.
- **connection** block****: Defines how Terraform connects to the remote machine. Can use:
 - **ssh** (for Linux/Unix)
 - **winrm** (for Windows)

Important Considerations

1. **Not Recommended for Production:** HashiCorp advises against using provisioners for production-grade infrastructure. Instead, use configuration management tools like Ansible, Chef, or cloud-init.
2. **Tightly Coupled:** Provisioners can make your Terraform code tightly coupled to the environment, reducing portability.
3. **Error Handling:** If a provisioner fails, Terraform marks the resource as "tainted", meaning it will be destroyed and recreated on the next apply.
4. **Security:** Be cautious with SSH keys and sensitive files.

Use Cases

- Bootstrapping a VM with config files
- Copying SSL certificates
- Uploading scripts before running them with remote-exec

Alternatives

- **cloud-init** (for Linux VMs)
- **User data scripts** (for AWS EC2)
- **Configuration management tools** (Ansible, Puppet, etc.)